Trabajo Práctico 6 — Guardar la partida

> **Diplomatura de Videojuegos · Semana 6** (clase 9) Objetivo: que el survivors del TP5 **recuerde cosas al cerrarse**. Primero, el récord en un archivo JSON. Después, un **ranking con nombres** en el mismo archivo. Por último, el desafío: una **estatua** que guarda la partida y un botón **Continuar** en el menú.

🎯 Al finalizar este TP

- `Partida` sabe **guardar** y **cargar** sus datos en `user://guardado.json`.
- El **récord** sobrevive a cerrar el juego.
- El menú pide un **nombre**, y el Game Over muestra un **top 5** que también se guarda.
- (Desafío) Una **estatua** que guarda vida, posición, tiempo y slimes en `user://checkpoint.json`, y un botón **Continuar** que vuelve a ese momento.

> 💡 **Tiempo estimado:** las partes 1 y 2 son lo de la clase (unos 60 min). La parte 3 es un desafío con pistas: lleva lo que cada uno decida. **Escribir el código a mano**, y abrir el archivo JSON cada vez que se guarda algo: es la mejor forma de ver si anda.

> 🔗 **Viene de:** la Clase 9 (archivos, `res://` y `user://`, JSON, `FileAccess`) y la Clase 6 (el Autoload `Partida`, el menú y el Game Over).

📍 Punto de partida

Este TP continúa el `tp5`. Copiar la carpeta del proyecto y renombrarla `tp6`, y abrirla con Godot.

Confirmar que el **punto de control 3 del TP5** sigue andando: al perder aparece el Game Over con el tiempo, los slimes y el récord, y **Reintentar** arranca de cero.

> 🧠 **Qué cambia y qué no.** El juego no se toca. Se modifican `partida.gd`, `menu.gd` y `game_over.gd`, y se suman dos nodos de UI. La estatua del desafío es la única escena nueva.

🧩 Cómo va a quedar el proyecto

```
partida.gd          Autoload "Partida"
                    + RUTA, nombre, ranking                    ← NUEVO
                    + guardar(), cargar(), agregar_al_ranking() ← NUEVO

menu.tscn           Menu (Control)                             menu.gd
  └── Botonera: Titulo, CampoNombre (LineEdit) ← NUEVO, BtnJugar, BtnSalir

game_over.tscn      GameOver (Control)                         game_over.gd
  └── Botonera: Titulo, LabelResultado, LabelRecord,
                LabelRanking (Label) ← NUEVO, BtnReintentar, BtnMenu

estatua.tscn        (desafío) Estatua (Area2D)                 estatua.gd

user://guardado.json      el récord y el ranking
user://checkpoint.json    (desafío) la foto de la partida
```

PARTE	QUÉ SE HACE	QUÉ SE VE
1	El récord en un archivo JSON	El récord sigue ahí al volver a abrir el juego
2	Ranking con nombre	El top 5 en el Game Over
3	Desafío: la estatua	"Continuar" vuelve al momento guardado

💾 Parte 1 — El récord en un archivo JSON

> **Concepto (Clase 9, bloques 1 y 2):** las variables viven en la **memoria** y se borran al cerrar el juego. Para que algo quede, hay que escribirlo en un **archivo**. Los archivos del juego van en `user://` : una carpeta del jugador donde se puede escribir, también en el `.exe` exportado. `res://` es la carpeta del proyecto y en el juego exportado es **solo lectura**.

1.1 · `guardar()` y `cargar()`

1. Abrir `partida.gd`. Arriba de todo, debajo de `extends Node`, agregar la ruta:

```
const RUTA = "user://guardado.json"
```

2. Al final del archivo, las dos funciones nuevas y un `_ready()`:

```
func _ready():
    cargar()

func guardar():
    var datos = {"mejor_tiempo": mejor_tiempo}
    var archivo = FileAccess.open(RUTA, FileAccess.WRITE)
    archivo.store_string(JSON.stringify(datos, "\t"))

func cargar():
    if not FileAccess.file_exists(RUTA):
        return
    var texto = FileAccess.get_file_as_string(RUTA)
    var datos = JSON.parse_string(texto)
    if datos == null:
        print("No se pudo leer ", RUTA)
        return
    mejor_tiempo = datos.get("mejor_tiempo", 0.0)
```

🧠 **Línea por línea.**

- `const` es una variable que **no cambia**. La ruta se escribe una sola vez y se usa en todos lados.
- `{"mejor_tiempo": mejor_tiempo}` es un **diccionario**: el mismo formato que un objeto JSON, con clave y valor.
- `FileAccess.open(RUTA, FileAccess.WRITE)` crea el archivo, o lo **vacía** si ya existía, y lo deja listo para escribir.
- `JSON.stringify(datos, "\t")` convierte el diccionario en texto JSON. El `"\t"` lo escribe con sangría, una clave por línea, para que se pueda leer.
- `JSON.parse_string(texto)` hace lo contrario: del texto, otra vez un diccionario. Si el archivo está roto, devuelve `null`.
- `datos.get("mejor_tiempo", 0.0)` devuelve el valor de la clave, o `0.0` si el archivo no la tiene.

🧠 **¿Por qué `cargar()` va en el `_ready()` de `Partida`?** Porque un Autoload se crea **antes que cualquier escena**. Cuando aparece el menú, el récord ya está leído.

1.2 · Guardar cuando hay récord

3. En `game_over.gd`, dentro del `if` del récord, agregar una línea:

```
    if Partida.tiempo > Partida.mejor_tiempo:
        Partida.mejor_tiempo = Partida.tiempo
        Partida.guardar()
        $Botonera/LabelRecord.text = "¡Nuevo récord!"
```

1.3 · Encontrar el archivo

4. Jugar con **F5** hasta perder. Tiene que decir **¡Nuevo récord!**
5. En el editor: **Project** → **Open User Data Folder**. Se abre una carpeta parecida a esta:

```
C:\Users\<usuario>\AppData\Roaming\Godot\app_userdata\tp6\
```

6. Abrir `guardado.json` con el Bloc de notas:

```
{
    "mejor_tiempo": 37.8166
}
```

✅ **Punto de control 1:** cerrar el juego, volver a abrirlo con **F5**, jugar una partida corta y perder: el Game Over dice **Récord: 37 s** (o el que se haya hecho), no "¡Nuevo récord!".

> 🧪 **Para probar:** con el juego cerrado, cambiar el número en el Bloc de notas por `999` y guardar. Al jugar, el récord es 999. Un archivo JSON se puede editar a mano: sirve para probar… y para hacer trampa.

🛟 **El récord no se guarda o no se carga**

<details>
<summary>Ver soluciones</summary>

- **No hay ningún archivo en la carpeta:** no se llamó a `Partida.guardar()`, o no hubo récord nuevo. Revisar que la línea esté **dentro** del `if`.
- **El archivo está, pero al abrir el juego el récord vuelve a 0:** falta el `_ready()` con `cargar()` en `partida.gd`.
- `No se pudo leer user://guardado.json` **en la consola:** el archivo se editó a mano y le falta una llave, una coma o una comilla. Borrarlo: el juego lo vuelve a crear.
- `Identifier "FileAccess" not declared`**:** está mal escrito. Es `FileAccess`, con F y A mayúsculas.
- **La carpeta del proyecto no se llama** `tp6`**:** el nombre de la carpeta de `user://` sale de **Project Settings** → **Application** → **Config** → **Name**, no de la carpeta del proyecto.

</details>

🏆 Parte 2 — Un ranking con nombre

> **Concepto (Clase 9, bloque 3):** un ranking es una **lista de objetos**, uno por partida. En JSON se escribe con corchetes, y en GDScript es un `Array` de diccionarios. Va en el **mismo archivo** que el récord: es una clave más.

```
{
    "mejor_tiempo": 80.5,
    "ranking": [
        { "kills": 31, "nombre": "Dani", "tiempo": 80 },
        { "kills": 20, "nombre": "Beto", "tiempo": 52 }
    ]
}
```

2.1 · El ranking en `Partida`

1. En `partida.gd`, dos variables nuevas debajo de `mejor_tiempo`:

```
var nombre = "Anónimo"
var ranking = []
```

2. En `guardar()`, el diccionario suma la clave `"ranking"`:

```
func guardar():
    var datos = {
        "mejor_tiempo": mejor_tiempo,
        "ranking": ranking
    }
    var archivo = FileAccess.open(RUTA, FileAccess.WRITE)
    archivo.store_string(JSON.stringify(datos, "\t"))
```

3. En `cargar()`, una línea al final:

```
    ranking = datos.get("ranking", [])
```

4. Dos funciones nuevas, al final de `partida.gd`:

```
func agregar_al_ranking():
    ranking.append({"nombre": nombre, "tiempo": int(tiempo), "kills": kills})
    ranking.sort_custom(mas_tiempo)
    ranking = ranking.slice(0, 5)

func mas_tiempo(a, b):
    return a["tiempo"] > b["tiempo"]
```

🧠 **Línea por línea.**

- `append()` suma la partida que terminó **al final** de la lista.
- `sort_custom(mas_tiempo)` ordena la lista usando una función propia. Godot le pasa dos filas, `a` y `b`, y la función responde si `a` va **antes** que `b`: acá, si duró más.
- `slice(0, 5)` devuelve una lista nueva con los elementos del 0 al 4: los cinco mejores. El sexto se descarta.
- `datos.get("ranking", [])`: el `guardado.json` de la Parte 1 **no tiene** la clave `"ranking"`. Con `datos["ranking"]`, el juego se cerraría con un error; con `get()`, empieza con una lista vacía.

2.2 · El nombre en el menú

5. Abrir `menu.tscn`. Hijo de `Botonera` → `LineEdit` → `CampoNombre`. Arrastrarlo para que quede **entre** `Titulo` **y** `BtnJugar`. En el Inspector: **Placeholder Text** = `Nombre`.
6. En `menu.gd`, en `_on_jugar()`, antes de cambiar de escena:

```
func _on_jugar():
    if $Botonera/CampoNombre.text != "":
        Partida.nombre = $Botonera/CampoNombre.text
    Partida.reiniciar()
    get_tree().change_scene_to_file("res://arena.tscn")
```

🧠 `LineEdit` es un campo de texto de una línea. Lo que se escribió está en su propiedad `text`. Si queda vacío, se mantiene el nombre anterior (o "Anónimo").

2.3 · El top 5 en el Game Over

7. Abrir `game_over.tscn`. Hijo de `Botonera` → `Label` → `LabelRanking`, debajo de `LabelRecord`. **Horizontal Alignment** = `Center`.
8. En `game_over.gd`, **sacar** el `Partida.guardar()` del `if` (el de la Parte 1) y, después del `if` / `else` del récord, agregar:

```
    Partida.agregar_al_ranking()
    Partida.guardar()
    mostrar_ranking()
```

9. Y la función nueva, al final del archivo:

```
func mostrar_ranking():
    var texto = "TOP 5\n"
    for i in Partida.ranking.size():
        var fila = Partida.ranking[i]
        texto += str(i + 1) + ". " + fila["nombre"]
        texto += " · " + str(int(fila["tiempo"])) + " s\n"
    $Botonera/LabelRanking.text = texto
```

🧠 **¿Por qué se guarda siempre y no solo con récord?** Porque ahora **cada partida** puede entrar al top 5, aunque no supere al primero.

🧠 `"\n"` es un salto de línea dentro de un texto: cada jugador queda en su renglón. `int(fila["tiempo"])` hace falta porque JSON no distingue enteros de decimales: se guardó `52` y vuelve `52.0`.

✅ **Punto de control 2:** jugar tres partidas con nombres distintos. El Game Over muestra, por ejemplo:

```
TOP 5
1. Beto · 52 s
2. Ana · 37 s
3. Caro · 12 s
```

Cerrar el juego, volver a abrirlo y perder otra vez: los tres siguen ahí, ordenados, con la partida nueva en su lugar. Con más de cinco partidas, solo quedan las cinco mejores. `guardado.json` tiene la lista completa.

🛟 **Errores comunes con el ranking**

▼ **Ver soluciones**

- `Invalid access to key 'ranking'`**:** en `cargar()` se usó `datos["ranking"]` en vez de `datos.get("ranking", [])`, y el archivo es el de la Parte 1.
- **El ranking muestra `52.0 s`:** falta el `int()` en `mostrar_ranking()`.
- **Todas las partidas dicen "Anónimo":** la ruta `$Botonera/CampoNombre` no coincide con el árbol, o el `if` está **después** del cambio de escena.
- **El ranking queda al revés:** en `mas_tiempo()` el signo es `>`. Con `<`, el primero es el que menos duró.
- **La misma partida aparece dos veces:** `agregar_al_ranking()` se llama en dos lugares. Va una sola vez, en el `_ready()` del Game Over.
- **El texto del ranking no entra en la pantalla:** bajar el **Font Size** de los `Label` o la **Separation** de la `Botonera`.

🗿 Parte 3 — Desafío: una estatua que guarda la partida

> **Concepto (Clase 9, bloque 4):** un **checkpoint** es una foto del momento: se guarda lo mínimo para **volver a armar** la partida, no todo lo que hay en pantalla. En el survivors, eso es la vida y la posición del caballero, el tiempo y los slimes eliminados. Los slimes en pantalla y las balas **no** se guardan: vuelven a aparecer solos.

Lo que se pide:

1. Una **estatua** en la arena. Cuando el caballero está cerca, aparece un cartel **E: guardar**. Al apretar **E**, se escribe `user://checkpoint.json` con vida, posición, tiempo y slimes, y el cartel dice **¡Guardado!**
2. En el menú, un botón **Continuar** que **solo se ve** si existe `checkpoint.json`.
3. **Continuar** abre la arena con el caballero en el lugar guardado, con su vida, y con el tiempo y los slimes como estaban.
4. **Una decisión de diseño**: al morir, ¿se borra el checkpoint (la próxima partida empieza de cero) o se conserva (se puede volver a la estatua)? Elegir una y anotarla en `aporte.md` o en un comentario.

Este desafío **no tiene paso a paso**. Abajo hay cinco pistas, de menor a mayor ayuda: abrir una sola por vez, y solo si hace falta.

<details>
<summary>▼ Pista 1 — La escena de la estatua</summary>

Es como la moneda del TP2: un `Area2D` que se entera de quién entra y sale.

```
Estatua  (Area2D)          estatua.gd
├── CollisionShape2D       ← un círculo grande, de unos 50 px: la zona "cerca"
├── ColorRect              ← gris, de 24 × 40 (o cualquier sprite)
└── Cartel  (Label)        ← "E: guardar", arriba de la estatua
```

Para la tecla, una acción nueva en el **Input Map**: `interactuar`, con la **E**.
</details>

<details>
<summary>▼ Pista 2 — Saber si el caballero está cerca</summary>

Una variable `jugador_cerca` que se pone en `true` con `body_entered` y en `false` con `body_exited` (las dos señales, conectadas en `_ready()`, solo si el cuerpo está en el grupo `"jugador"`). El cartel se muestra y se oculta con `$Cartel.visible`.

En `_process()`: si `jugador_cerca` y `Input.is_action_just_pressed("interactuar")`, se guarda.
</details>

▼ Pista 3 — Qué escribir en el archivo

Una función nueva en `Partida`, con su propia ruta. Recibe al jugador para leer su vida y su posición. JSON no conoce `Vector2`: la posición va en dos números.

```
const RUTA_CHECKPOINT = "user://checkpoint.json"

func guardar_checkpoint(jugador):
    var datos = {
        "vida": jugador.vida,
        "x": jugador.position.x,
        "y": jugador.position.y,
        "tiempo": tiempo,
        "kills": kills
    }
    # ... y el resto, igual que en guardar()
```

La estatua la llama así:
`Partida.guardar_checkpoint(get_tree().get_first_node_in_group("jugador"))`.

▼ Pista 4 — El botón Continuar

Un `Button` **`BtnContinuar`** en el menú, arriba de Jugar. En `_ready()` de `menu.gd`:

```
    $Botonera/BtnContinuar.visible = FileAccess.file_exists(Partida.RUTA_CHECKPOINT)
```

Al apretarlo, `Partida` lee el archivo y **guarda el diccionario en una variable** (por ejemplo, `var checkpoint = null`), pone `tiempo` y `kills` con lo leído (`int()` para los slimes) y recién ahí se cambia a la arena. La arena todavía no existe cuando se aprieta el botón: por eso el diccionario espera en `Partida`.

▼ **Pista 5 — Aplicarlo en la arena**

En `arena.gd`, un `_ready()`. El `_ready()` de un nodo corre **después** del de todos sus hijos: cuando corre el de la `Arena`, el HUD ya existe y el caballero ya empezó con vida 100. Es el momento justo para pisar esos valores.

```
func _ready():
    if Partida.checkpoint != null:
        var datos = Partida.checkpoint
        $Jugador.vida = int(datos["vida"])
        $Jugador.position = Vector2(datos["x"], datos["y"])
        $HUD.actualizar_vida($Jugador.vida)
        Partida.checkpoint = null
```

¿Por qué `Partida.checkpoint = null` al final? Sin esa línea, el siguiente **Reintentar** también volvería a la estatua.

Para borrar el archivo, si esa fue la decisión de diseño:
`DirAccess.remove_absolute(Partida.RUTA_CHECKPOINT)`.

✅ **Punto de control 3:** jugar, acercarse a la estatua y apretar **E** con, por ejemplo, 65 de vida y 22 slimes. Seguir jugando y **cerrar el juego**. Al abrirlo, el menú muestra **Continuar**; al apretarlo, el caballero aparece junto a la estatua con la barra en 65, y el tiempo y los slimes siguen desde donde estaban. `checkpoint.json` se puede leer con el Bloc de notas.

🛟 **Errores comunes con el checkpoint**

▼ **Ver soluciones**

- **El cartel no aparece:** el caballero no está en el grupo `"jugador"` cuando entra, o el `CollisionShape2D` de la estatua no tiene forma.
- **E no hace nada:** la acción `interactuar` no existe en el Input Map, o está escrita distinto.
- **Continuar no aparece nunca:** se pregunta por la ruta de `guardado.json` en vez de `checkpoint.json`.
- **El caballero aparece donde siempre:** la posición se aplica en el `_ready()` del **Jugador**, o `Partida.checkpoint` ya estaba en `null`.
- **La barra muestra 100 aunque la vida es 65:** falta `$HUD.actualizar_vida(...)`, o se llama antes de que el HUD ponga su 100.
- **Error de tipos al cargar la posición (`String` en vez de `Vector2`):** se guardó `jugador.position` entero en vez de `x` e `y`. JSON lo escribe como el texto `"(320.5, 180.0)"` y no vuelve a ser un `Vector2`.

Entrega

Entregar la **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`). Si se hizo el desafío, anotar en `aporte.md` la decisión de la Parte 3.4.

Nombre: `tp6-ApellidoNombre.zip`

Checklist de autoevaluación

- [ ] `partida.gd` tiene `guardar()` y `cargar()`, y `cargar()` se llama en su `_ready()`.
- [ ] Todo se guarda en `user://`, nunca en `res://`.
- [ ] El récord sigue ahí después de cerrar y abrir el juego.
- [ ] `cargar()` usa `datos.get(...)` y no se rompe si falta una clave o no hay archivo.
- [ ] El menú tiene un campo para el **nombre**.
- [ ] El Game Over muestra un **top 5** ordenado, sin `.0`, que se conserva al cerrar el juego.
- [ ] `guardado.json` se puede abrir y leer con el Bloc de notas.
- [ ] (Desafío) La estatua guarda con **E**, y **Continuar** vuelve a ese momento.

Código completo de referencia

Las partes 1 y 2. **El desafío no tiene solución publicada:** las pistas alcanzan.

▼ partida.gd

```
extends Node

const RUTA = "user://guardado.json"

var kills = 0
var tiempo = 0.0
var mejor_tiempo = 0.0
var nombre = "Anónimo"
var ranking = []

func _ready():
    cargar()

func reiniciar():
    kills = 0
    tiempo = 0.0

func guardar():
    var datos = {
        "mejor_tiempo": mejor_tiempo,
        "ranking": ranking
    }
    var archivo = FileAccess.open(RUTA, FileAccess.WRITE)
    archivo.store_string(JSON.stringify(datos, "\t"))

func cargar():
    if not FileAccess.file_exists(RUTA):
        return
    var texto = FileAccess.get_file_as_string(RUTA)
    var datos = JSON.parse_string(texto)
    if datos == null:
        print("No se pudo leer ", RUTA)
        return
    mejor_tiempo = datos.get("mejor_tiempo", 0.0)
    ranking = datos.get("ranking", [])

func agregar_al_ranking():
    ranking.append({"nombre": nombre, "tiempo": int(tiempo), "kills": kills})
    ranking.sort_custom(mas_tiempo)
    ranking = ranking.slice(0, 5)

func mas_tiempo(a, b):
    return a["tiempo"] > b["tiempo"]
```

▼ `menu.gd` y `game_over.gd`

```
# menu.gd
extends Control

func _ready():
    $Botonera/BtnJugar.pressed.connect(_on_jugar)
    $Botonera/BtnSalir.pressed.connect(_on_salir)
    $Botonera/BtnJugar.grab_focus()

func _on_jugar():
    if $Botonera/CampoNombre.text != "":
        Partida.nombre = $Botonera/CampoNombre.text
    Partida.reiniciar()
    get_tree().change_scene_to_file("res://arena.tscn")

func _on_salir():
    get_tree().quit()
```

```
# game_over.gd
extends Control

func _ready():
    var segundos = str(int(Partida.tiempo))
    $Botonera/LabelResultado.text = "Tiempo: " + segundos + " s · Slimes eliminados: " +
      str(Partida.kills)
    if Partida.tiempo > Partida.mejor_tiempo:
        Partida.mejor_tiempo = Partida.tiempo
        $Botonera/LabelRecord.text = "¡Nuevo récord!"
    else:
        $Botonera/LabelRecord.text = "Récord: " + str(int(Partida.mejor_tiempo)) + " s"
    Partida.agregar_al_ranking()
    Partida.guardar()
    mostrar_ranking()
    $Botonera/BtnReintentar.pressed.connect(_on_reintentar)
    $Botonera/BtnMenu.pressed.connect(_on_menu)
    $Botonera/BtnReintentar.grab_focus()

func mostrar_ranking():
    var texto = "TOP 5\n"
    for i in Partida.ranking.size():
        var fila = Partida.ranking[i]
        texto += str(i + 1) + ". " + fila["nombre"]
        texto += " · " + str(int(fila["tiempo"])) + " s\n"
    $Botonera/LabelRanking.text = texto

func _on_reintentar():
    Partida.reiniciar()
    get_tree().change_scene_to_file("res://arena.tscn")

func _on_menu():
    get_tree().change_scene_to_file("res://menu.tscn")
```

🌟 Extra (opcional, para los que quieran más)

- **El ranking en el menú.** Un `Label` en el menú que muestre el top 5 antes de jugar. La función `mostrar_ranking()` del Game Over sirve casi igual.
- **Borrar el ranking.** Un botón en el menú que haga `Partida.ranking = []`, `Partida.mejor_tiempo = 0.0` y `Partida.guardar()`.
- **Guardar en binario.** Cambiar `guardar()` y `cargar()` para usar `archivo.store_var(datos)` y `archivo.get_var()`, con `FileAccess.READ` para leer, en `user://guardado.dat`. Comparar: ¿se puede leer con el Bloc de notas? ¿Hace falta el `int()`?
- **Una estatua que se gana.** Que la estatua no esté desde el principio: aparece cada 60 segundos en un lugar al azar y se va a los 10. Guardar se vuelve una decisión arriesgada.

- **Guardar cuesta algo.** Como la cinta de *Resident Evil*: la estatua solo guarda si se eliminaron 20 slimes desde el último guardado. Mostrar cuántos faltan en el cartel.
- **Autoguardado.** Un `Timer` en la arena que llame a `guardar_checkpoint()` cada 30 segundos, con un aviso de "Guardando…" en el HUD durante un segundo.

📚 Recursos

- Rutas: **File paths in Godot projects**
- Leer y escribir archivos: **FileAccess**
- JSON: **JSON** y **Dictionary**
- Guardar escenas enteras, para más adelante: **Saving games**. Usa la misma idea del checkpoint (un diccionario por nodo, pasado a JSON) para muchos objetos a la vez.